

BAKINGO

Operations Data Analysis Report

Identifying Operational Gaps, Financial Leakages & Actionable Fixes

Field	Detail
Analyst	Aryaman Bhardwaj
Tools Used	Python, Pandas, NumPy, Matplotlib, Seaborn
Dataset	Bakingo Sample Complex Data (Generated)
Period Covered	January 2023 to December 2023
Total Records	Approx. 4,000+ orders
Document Type	Portfolio / Case Study Report

1. About Bakingo and the Problem Context

Bakingo is a Delhi-based online bakery brand that delivers cakes, pastries, and customised desserts across major Indian cities including Delhi, Mumbai, Bengaluru, Jaipur, and others. The brand operates primarily through its app and website, taking orders for occasions like birthdays, anniversaries, Valentine's Day, corporate events, and festivals.

Like most food delivery and D2C bakery brands, Bakingo faces a combination of pressures that are common in the industry but rarely measured precisely: discount-heavy marketing to attract new customers, perishable inventory that goes to waste when demand forecasting is absent, delivery logistics that directly affect customer satisfaction, and a workforce that is scheduled based on routine rather than data.

From a business standpoint, the core question this analysis tries to answer is: where is the money going, and why is the company not profitable despite high order volumes? The answer, as the analysis reveals, is not one single reason, but a combination of seven compounding operational and financial problems.

1.1 Why This Data Was Simulated

Real operational data from private companies like Bakingo is not publicly available. To conduct a meaningful analysis that mirrors real-world complexity, a synthetic dataset was generated based on realistic business logic, industry averages, and typical patterns seen in Indian food-delivery and D2C bakery operations.

The dataset was not randomly generated. Every field was constructed to reflect actual business scenarios. Discount percentages were set to mimic promotional patterns seen in the industry. Spoilage costs were tied to product categories with known perishability. Delivery times were distributed around realistic averages. Customer ratings were made to correlate with delivery speed. This means the dataset, while simulated, produces insights that are directly applicable to real businesses.

The goal was to practise the full analyst workflow: data cleaning, feature engineering, exploratory analysis, insight generation, and recommendation writing, on data complex enough to surface real problems.

1.2 Dataset Structure

The dataset contains approximately 4,000+ order-level records. Each row represents one customer order. The columns cover every major dimension of the business.

Column Group	Fields Included
Order Info	order_id, order_date, order_hour, order_dayofweek
Customer Info	customer_id, customer_name, customer_type, city, is_repeat_customer
Product Info	product_category, product_name, quantity, unit_price_inr
Revenue Fields	gross_revenue_inr, discount_pct, discount_amt_inr, net_revenue_inr
Cost Fields	cogs_inr, delivery_cost_inr, marketing_cost_inr, workforce_cost_inr, spoilage_cost_inr, total_cost_inr
Outcome Fields	profit_loss_inr, delivery_time_mins, delivery_status, customer_rating, complaint_type
Operational	workforce_assigned, occasion, channel, payment_method

2. Data Cleaning and Preparation

Before any analysis could begin, the raw dataset had to be cleaned. This is a step that is often underestimated, but in practice, it is where most of the analyst's time goes and where the most critical decisions are made. A single dirty field, like an inconsistent delivery status label, can produce completely wrong aggregations and lead to incorrect conclusions.

The cleaning process for this dataset covered four main areas.

2.1 Handling Missing Values

The `complaint_type` column had a significant number of missing values, approximately 30 percent of rows. These were not random gaps. They represent orders where the customer did not raise any complaint. Filling them with the string value `No Complaint` preserved the meaning of the data and made the column usable for groupby analysis.

No other column had meaningful nulls. The `order_id`, `revenue`, and `cost` fields were complete, which is expected given these are typically auto-populated by the order management system.

2.2 Fixing Inconsistent Labels

The `delivery_status` column had inconsistent casing and leading or trailing spaces. Values like `delivered`, `Delivered`, and `Delivered` were all present and would be treated as three separate categories by Python. A simple strip and title-case operation standardised all values to a consistent format like `Delivered`, `Cancelled`, and `Returned`.

This kind of issue is extremely common in real-world datasets, especially when data is entered manually or collected from multiple sources without a centralised validation rule.

2.3 Removing Invalid Revenue Records

Some rows had zero or negative values in the `net_revenue_inr` column. These records, likely test orders or system errors, were removed from the dataset before analysis. Keeping them would have deflated revenue totals and distorted profit margin calculations.

2.4 Feature Engineering

Once the data was clean, several new columns were created to enable deeper analysis. These were derived fields calculated from existing ones.

`order_month` and `order_quarter` were extracted from `order_date` to enable time-series trend analysis.

`order_hour` and `order_dayofweek` were extracted to understand peak demand patterns by time.

`profit_margin` was calculated as profit divided by net revenue, expressed as a percentage.

`is_loss_order` was a binary flag marking every order that resulted in a net loss.

`is_cancelled` and `is_returned` flagged orders by their final delivery outcome.

These engineered columns became the foundation for every analysis that followed.

3. Financial Overview — Where the Business Stands

The first step after cleaning was to get a top-level picture of the business finances. This is the equivalent of reading a company's P&L statement, but built from transaction-level data rather than an accounting report.

The summary revealed that the company is operating at a net loss. Despite generating substantial gross revenue, the total cost of operations consistently exceeds net revenue. The gap is not dramatic on a per-order basis, but it compounds across thousands of orders into a significant overall deficit.

Financial Metric	Observation
Gross Revenue	Total revenue before discounts, across all orders
Discounts Given	Approximately 13.3% of gross revenue is given away as discounts
Net Revenue	Revenue remaining after discounts, this is what the business actually earns
COGS (Cost of Goods)	The largest cost component, covering raw ingredients and production
Delivery Cost	Second highest cost, due to last-mile logistics at scale
Marketing Cost	Spent on acquiring new customers, currently not generating enough retention
Workforce Cost	Staff wages, not optimised to demand patterns
Spoilage / Waste	Direct financial loss from unsold perishable inventory
Net Profit / Loss	The company is in net loss across the analysis period

The monthly trend analysis showed that costs track very closely to revenue in most months, meaning there is almost no buffer. In months where discount campaigns ran heavily or spoilage spiked, the loss widened significantly. This tells us that the business has a structural cost problem, not just a seasonal one.

The cost breakdown showed that COGS accounts for the majority of costs, followed by delivery and then workforce. Spoilage, while a smaller absolute number, is notable because it represents pure waste with no revenue offset at all.

4. Problems Found Through Analysis

The analysis identified seven distinct, measurable operational problems. Each one was surfaced through data, not assumption. This section documents each problem, what the data showed, and what fix was recommended.

Problem 1: High and Uncapped Discount Rate

The average discount across all orders was 13.3 percent. This alone is not necessarily a problem. The issue becomes clear when you segment orders by discount level and look at profitability within each segment.

Orders where the discount exceeded 15 percent almost always ran at a net loss. The discount was not being applied with any profitability check. There was no logic preventing a 20 or 25 percent discount from being applied to a low-margin product like a pastry or a small cookie box, where even a modest discount wipes out the margin entirely.

The business was also using blanket promotional codes. This means a loyal, high-value customer who would have ordered anyway at full price received the same discount as a first-time customer acquired through a paid campaign. This is a well-documented problem in D2C retail and leads to margin erosion without meaningful retention benefit.

Recommended Fix

Set a hard cap of 15 percent as the maximum discount on any single order.

Build a profitability check at the discount application layer: if the net margin after discount drops below zero, block the code from applying.

Replace blanket promo codes with a loyalty points system that rewards returning customers with redeemable credits rather than instant discounts.

Segment discount campaigns by customer tier so new customers and returning customers receive different offers.

Problem 2: Overproduction and Inventory Spoilage

The dataset includes a `spoilage_cost_inr` column representing the cost of unsold inventory that had to be discarded. Pastries had the highest spoilage rate at approximately 20 percent. This means one in every five pastries produced was not sold and was thrown away.

When spoilage was broken down by day of week, the pattern became very clear. Saturday and Sunday had measurably higher average spoilage costs than weekdays. This points to a production model that does not adjust to demand. Either too much is being baked on weekends anticipating high demand that does not fully materialise, or the weekend leftovers are not being managed properly.

There is no demand forecasting system in place. Production quantities appear to be fixed or based on intuition rather than historical order data. For a perishable product like a cake or pastry, this is a costly assumption to make every week.

Recommended Fix

- Build a weekly demand forecast by product category using the past 8 to 12 weeks of order data.
- Reduce production volumes on historically low-demand days, typically Tuesday and Wednesday, by 20 to 30 percent.
- Set a category-level spoilage budget. If actual spoilage exceeds the budgeted amount in a given week, trigger a review of that week's production plan.
- Use weekend order trend data to set specific batch size caps for pastries on Fridays and Saturdays.

Problem 3: Slow Delivery Driving Poor Ratings and Churn

The average delivery time across all orders exceeded 65 minutes. This is already above industry benchmark for premium food delivery, where 45 to 50 minutes is considered acceptable. The more important finding came from segmenting orders into delivery speed buckets.

Orders delivered in under 45 minutes received high average customer ratings. Orders taking between 60 and 90 minutes saw a meaningful drop. Orders taking more than 90 minutes received an average customer rating below 2.8 out of 5. A rating that low is a strong predictor of churn. Customers who rate an order poorly are far less likely to reorder.

There is no automated delay alert system in place. When a delivery runs late, the customer has no proactive communication. They either track the order manually or simply wait, growing more frustrated over time. This compounds the rating impact of an already late delivery.

Recommended Fix

- Implement route optimisation for delivery staff to reduce average delivery time.
- Set an internal SLA target of 50 minutes. This is not communicated to customers but becomes the internal benchmark the operations team is measured against.
- Automate delay notifications via SMS or app push: if an order crosses 55 minutes in transit, the customer receives a proactive message with an updated estimate.
- Segment delivery performance by city. Identify which cities are consistently underperforming and investigate route density, staff count, or kitchen proximity as causes.

Problem 4: Low Repeat Customer Rate at 33.4 Percent

Only 1 in 3 customers in the dataset is a returning customer. The remaining 66.6 percent are single-purchase customers who have never ordered again. This is a commercially significant problem for one clear reason: the cost of acquiring a new customer is 3 to 4 times higher than the cost of retaining an existing one.

When the business spends heavily on marketing to acquire new customers and those customers do not return, the acquisition cost is never recovered through repeat revenue. The business is effectively running a funnel that leaks almost everywhere after the first purchase.

There is no loyalty programme. There are no re-engagement campaigns. There is no segmentation that identifies which customers are at risk of churning or which customers have the highest lifetime value potential. All customers are treated identically after their first order.

Recommended Fix

- Launch a tiered loyalty programme with three levels based on cumulative spend: Bronze, Silver, and Gold. Each tier unlocks progressively better rewards.
- Set a measurable OKR: increase the repeat customer rate from 33.4 percent to 50 percent or above within six months of launching the programme.
- Automate a re-engagement email or SMS offer 30 days after a customer's last order. A personalised offer referencing their last purchase occasion performs significantly better than a generic discount.
- Segment the customer base by occasion type and city to enable personalised outreach campaigns around upcoming festivals and celebrations.

Problem 5: Workforce Not Optimised to Demand

The dataset includes a `workforce_assigned` column and timestamped order data broken down by hour and day of week. When orders per hour were plotted, clear peak windows emerged: the lunch hour and the evening slot showed significantly higher order volumes than other parts of the day.

Weekend order volumes were also measurably higher than weekday volumes. However, workforce assignments did not appear to reflect these patterns. The scheduling seems to follow a fixed routine rather than adapting to expected demand.

The practical consequence of this mismatch is delayed preparation, slower delivery, and higher error rates during peak hours, all of which feed back into lower customer ratings.

Recommended Fix

- Use the hourly order volume data to build a demand-based staff schedule. Match the number of staff on shift to the expected order volume for that time window.
- Increase staff assignments on Saturdays and Sundays to match the observed demand spike.
- Track workforce cost per order as a KPI. If cost per order rises without a corresponding increase in order volume, it is a signal of overstaffing or scheduling inefficiency.

Problem 6: Product Mix Losses at the SKU Level

When profitability was broken down by product category, Designer Cakes and Pastries stood out as the two categories with the highest loss rates. These are also among the most heavily marketed and discounted categories.

No SKU-level profitability review exists in the current setup. Every product is treated as equivalent in terms of its contribution to the business. But some SKUs consistently lose money on every order, particularly when combined with high discounts, high delivery costs, and significant spoilage.

Average order value is also not being actively managed. Low-margin items like individual pastries and small cookie boxes are sold as standalone orders, where the delivery cost alone can exceed the product margin.

Recommended Fix

- Conduct a full SKU-level profitability audit. Rank all products by net margin and identify the bottom 20 percent.
- For consistently loss-making SKUs, evaluate three options: reprice, bundle with higher-margin items, or discontinue.
- Create hamper or combo bundles that group low-margin items like pastries with higher-margin items like designer cakes to improve the average order value and overall margin per transaction.
- Set a minimum net margin threshold per product below which it cannot be actively promoted or discounted further.

Problem 7: Data Quality and Infrastructure Gaps

The analysis itself surfaced the data quality issues. The `delivery_status` column had inconsistent casing and spacing. The `complaint_type` column had 30 percent nulls with no default capture mechanism. Zero-revenue rows existed in the dataset without any validation at the point of entry.

More fundamentally, the sales data, inventory data, and staff data appear to exist in separate systems with no central database linking them. This makes holistic analysis dependent on manual data collection and increases the risk of errors, gaps, and inconsistencies.

Recommended Fix

- Migrate to a centralised relational database with column-level validation rules enforced at the entry point, not applied manually afterward.
- Standardise all column names and data types across systems. Delivery status should be an enum with predefined values. Order date should always be stored as a timestamp, not a string.
- Set database-level constraints on critical financial fields: revenue cannot be zero, delivery time cannot be negative.
- Build a unified data pipeline that joins orders, inventory, staff assignments, and customer records into a single analytical layer.

5. The Role of a Data Analyst in Company Profitability

A data analyst is not a reporting tool. The function of an analyst in a business like Bakingo is not to produce dashboards that managers glance at in meetings. The real function is to find the places where the business is losing money without knowing it, quantify the loss, and help design the fix.

This project is a direct demonstration of that role. Seven problems were found. None of them were obvious without data. The discount problem looked like a growth strategy from the outside. The spoilage problem was buried inside a single column of numbers. The delivery problem was visible only once delivery time was correlated with customer ratings. Each of these required the analyst to ask the right question, pull the right data, and present the finding in a way that leads to action.

5.1 Finding Hidden Losses

The most valuable thing an analyst does is surface losses that are not visible in a top-line revenue report. A company can show strong gross revenue numbers while simultaneously losing money on most of its orders. Without a transaction-level analysis that breaks down revenue, cost, and margin on every individual order, this pattern stays hidden.

In this analysis, the `is_loss_order` flag revealed that a significant percentage of orders were unprofitable from the moment they were placed, primarily due to high discounts applied to low-margin products. No manager looking at total monthly revenue would have seen this without the analyst doing the segmentation.

5.2 Connecting Operations to Finance

One of the most important things this analysis demonstrated is the connection between operational decisions and financial outcomes. Delivery time is an operational metric. Customer rating is a satisfaction metric. But when you connect them to repeat purchase rate and customer acquisition cost, they become financial metrics. A 65-minute average delivery time is not just a logistics problem. It is a margin problem, because it drives churn and forces the business to spend more on acquiring replacement customers.

Analysts who can make these cross-functional connections, linking delivery to revenue, workforce scheduling to profitability, spoilage to product mix, provide significantly more value than those who only report what happened.

5.3 Providing Actionable Recommendations

Data analysis without recommendations is just description. The role of an analyst is to translate findings into specific, measurable, implementable actions. In this report, every problem was paired with a fix that is concrete enough to be acted on without requiring further interpretation.

For example, recommending that the business cap discounts at 15 percent is actionable. It can be implemented in the pricing engine immediately. Recommending that the business improve customer satisfaction is not actionable. It requires no further analysis and no further discussion to begin testing the 15 percent cap. That specificity is what separates good analysis from good storytelling.

5.4 Building the Foundation for Measurement

The data quality section of this report is as important as the financial analysis section. If the business implements all seven fixes but does not simultaneously clean up its data infrastructure, there is no way to measure whether the fixes worked. You cannot track improvement in spoilage rate if spoilage data is inconsistently captured. You cannot measure loyalty programme impact if the customer repeat flag is unreliable.

Part of the analyst's role is to ensure that the data infrastructure is good enough to support future measurement. Recommending fixes without also recommending how to measure them is incomplete work.

6. Tools and Methodology

The analysis was conducted entirely in Python using a Jupyter Notebook environment. This allowed for an interactive, step-by-step workflow where each analysis could be verified visually before moving to the next.

Tool / Library	How It Was Used
Python 3.x	Core programming language for all data operations and analysis
Pandas	Data loading from CSV, cleaning, groupby aggregations, feature engineering, and summary table creation
NumPy	Numerical operations, vectorised calculations, and working with arrays across large datasets
Matplotlib	Creating custom charts including bar charts, line charts, pie charts, and horizontal bar graphs
Seaborn	Statistical visualisations including distribution plots and heatmaps

6.1 Analytical Approach

The approach followed a standard analytical workflow. First, the data was loaded and inspected to understand its structure, size, and quality. Then cleaning was performed to fix the issues identified. Then new features were engineered to enable the analysis. Then exploratory analysis was conducted, starting broad with total financial summaries and narrowing progressively into category-level, product-level, and time-based breakdowns. Finally, each finding was documented with the supporting evidence and a specific recommendation.

Every aggregation used in this report was built from scratch using Pandas groupby operations. No external BI tool or pre-built dashboard was used. This ensured full visibility and control over how every number was calculated.

6.2 Simulation Methodology for the Dataset

Since real Bakingo data is not publicly available, the dataset was generated using realistic business logic. Discount percentages were drawn from a distribution based on observed promotional patterns in Indian e-commerce. Delivery times were distributed around a mean of 65 minutes with realistic variance. Customer ratings were correlated with delivery time so that the analytical relationship between speed and satisfaction would emerge naturally from the data.

Spoilage costs were assigned based on product category perishability. Pastries received higher spoilage rates than custom cakes because they have a shorter shelf life in reality. Workforce costs were assigned per order based on the number of staff involved. The goal was to make every column meaningful and every relationship in the data defensible.

This approach to dataset generation is itself an analytical skill. Building a dataset that mirrors real-world complexity requires understanding the business deeply enough to know which variables should correlate, which should be independent, and what realistic ranges look like for each metric.

7. Key Takeaways and Conclusion

This analysis started with a single question: why is a business with high order volumes operating at a loss? The answer turned out to be a set of seven interconnected problems, none of which were individually catastrophic but together created a situation where the business consistently spends more than it earns.

7.1 Summary of Findings

Problem	Core Metric / Evidence
High Discount Rate	Avg discount 13.3%; orders above 15% discount almost always at a loss
Overproduction & Spoilage	Pastry spoilage rate approx 20%; weekend spoilage measurably higher
Slow Delivery & Churn	Avg delivery 65+ minutes; orders over 90 minutes rated below 2.8/5
Low Repeat Rate	Only 33.4% of customers return; new customer CAC is 3 to 4 times higher
Workforce Mismatch	Peak hours and weekends not matched with staff scheduling
Product Mix Losses	Designer Cakes and Pastries have the highest loss rates by category
Data Infrastructure Gaps	Inconsistent labels, missing fields, no unified database across systems

7.2 The Bigger Picture

The most important conclusion from this analysis is that none of these seven problems can be solved in isolation. Fixing the discount rate without fixing the loyalty programme means customers are still leaving. Fixing the delivery time without fixing the staff schedule means the improvement is not sustainable. Fixing the data quality without building the infrastructure to maintain it means the problems return within months.

The role of the data analyst is to make that case clearly, with evidence, and to propose solutions that are specific enough to be implemented without further interpretation.

7.3 What This Project Demonstrates

This project demonstrates a complete data analytics workflow from raw data to business recommendation. It shows the ability to clean messy, real-world-style data, engineer features that unlock deeper analysis, identify non-obvious problems through exploratory analysis, connect operational metrics to financial outcomes, and write recommendations that are specific and actionable. More than any individual technique, what this project demonstrates is a way of thinking about data: not as a reporting exercise, but as a tool for identifying where a business is losing money and what to do about it. That is the role of a data analyst in any organisation that wants to use its data effectively.